The Emergent State Machine: An Architectural Framework for Deterministic, Interpretable, Governable Decision Systems

Richard Hines
Independent Researcher
Tacoma, Washington, USA
richardhinestacoma@gmail.com

Abstract

The Emergent State Machine (ESM) is a deterministic architectural framework for interpretable situational reasoning in complex domains. The architecture structures reasoning as a sequence of explicit, inspectable steps in which situational meaning is constructed as explicit state prior to any consideration of action.

Within each turn, raw observations are transformed into signals, which are assembled into a coherent, committed state representation of the current situation. This state is then expressed in policy-relevant coordinates through projection, evaluated for relevance via explicit gating, and, when warranted, acted upon through deterministic policy. This sequence enforces a structural separation between state construction (interpretation), projection, relevance gating, and authoritative policy action. Traceability is formalized as a causal chain $r_t \leftarrow g_t \leftarrow y_t \leftarrow x_t \leftarrow S_t \leftarrow o_t$, making each decision reconstructable from its underlying observations, signals, and policy logic.

By organizing behavior as a sequence of turns, the ESM makes state evolution visible as trajectories through interpretable and policy-relevant state spaces, so that longer-run behavior can be analyzed in terms of policy-defined desirable and undesirable regions, without implying global trajectory optimization.

The architecture establishes a deterministic boundary at the point of decision, ensuring that outcomes, including explicit no-op decisions, arise only from authorized policy evaluation over a fully constructed situational state. Because signals, projections, gating logic, and policies are explicitly defined and versioned, system behavior can evolve only through intentional, versioned modification of these architectural artifacts. The reasoning that produces each decision is explicitly structured, inspectable, and accountable, rather than an opaque internal process. This approach addresses a central limitation of contemporary decision systems, in which responsibility for consequential decisions remains with human operators while the computational processes supporting those decisions often remain uninspectable.

The canonical specification of the architecture is available at the ESM specification repository on GitHub[1].

Contributions

This paper introduces the Emergent State Machine (ESM), a deterministic architectural framework for structuring interpretable situational reasoning. The architecture formalizes reasoning as a sequence of turns in which observations are transformed into signals and assembled into a coherent, committed state representation prior to any evaluation of action. Decisions arise only after this state is fully constructed, and outcomes, including explicit no-op decisions, are determined exclusively through policy.

A central contribution of the architecture is the explicit separation between state construction (interpretation), projection, relevance gating, and policy action. This separation constrains where and how state mutation can occur, ensuring that interpretation is completed before decision evaluation and that policy is the sole mechanism by which action is authorized. As a result, system behavior is traceable to explicit architectural components rather than embedded within opaque transformations.

The framework further formalizes system behavior as trajectories through interpretable and policy-relevant state spaces over discrete turns. Policy-defined regions of desirability provide a way to describe longer-run behavior in terms of where trajectories tend to remain or return, without positing global optimization.

Because signals, projections, relevance determination logic, and policies are explicitly defined and versioned, the architecture supports Instrumented Deterministic Evolution, in which behavioral change occurs only through intentional modification of versioned system components. This enables systematic testing, comparison, and refinement of system behavior over time.

Finally, the architecture supports composition across temporal scales via layered ESMs and explicit state handoff, allowing multiple interacting systems to maintain local reasoning, shared context, and deterministic authority without loss of modularity or provenance.

1 Introduction

Contemporary decision systems often operate as opaque pipelines where complex transformations produce outputs without exposing reasoning steps. In domains like clinical care, education, and operational control, professionals bear responsibility for consequential decisions while computational support remains uninspectable. Threshold alerting surfaces measurements but leaves operators to reconstruct situational context themselves [4].

[1] `https://github.com/emergent-state-machine/esm-spec`

Observations $\longrightarrow$ **Signals** $\longrightarrow$ **Coherent State Vector** $\longrightarrow$ **Projection** $\longrightarrow$ **Relevance** $\longrightarrow$ **Policy** $\longrightarrow$ **Outcome**

Figure 1: shows canonical pipeline. Subsequent sections detail: signals (S_t), committed state (x_t), projection ($y_t = P(x_t)$), gating ($g_t = G(y_t)$), policy ($\pi(y_t)$).

The Emergent State Machine (ESM) structures reasoning as discrete *turns*, each committing to a coherent state x_t before any action consideration. Figure 2 shows the canonical pipeline: observations $o_t \to$ signals $S_t \to$ committed state $x_t \to$ projection $y_t \to$ gating $g_t \to$ policy outcome r_t.

Each turn formalizes the causal chain

$$r_t \leftarrow g_t \leftarrow y_t \leftarrow x_t \leftarrow S_t \leftarrow o_t.$$

State construction completes interpretation as a fully resolved situational construct. Projection re-expresses this state in policy coordinates without semantic alteration. Gating authorizes policy evaluation; policy alone produces outcomes (including explicit no-op). Interpretation precedes authorization precedes action.

System evolution manifests as trajectories $\{x_1, x_2, \dots\}$ through policy-relevant state space, arising from locally governed turns rather than global optimization. Time advances discretely via completed commitments, with turns occurring at decision-relevant moments rather than uniform observation intervals.

Key properties:

- **Committed turns**: Complete reasoning frames, inspectable in isolation or sequence
- **Deterministic boundary**: Policy is sole action authority, activated only by gating
- **Explicit artifacts**: Signals, projections, gating, policies are versioned and auditable
- **Instrumented evolution**: Behavioral change occurs only through governed revision
- **Layered composition**: ESMs interoperate via explicit state handoff (Section 10)

This yields computational reasoning where every outcome traces to specific state, authorization boundary, and policy evaluation. The canonical specification—defining deterministic execution such that identical inputs yield identical outcomes—is at github.com/emergent-state-machine/esm-spec. A reference implementation is under active development.

2 ESM as a Discrete-Time System

The ESM operates as a discrete-time system where time advances through completed turns. Each turn T_t constitutes a full reasoning frame, committing to state x_t before any action consideration.

At turn t, observations o_t produce signals $S_t \to$ committed state $x_t \to$ projection $y_t \to$ gating $g_t \to$ policy outcome r_t, formalized as the causal chain:

$$r_t \leftarrow g_t \leftarrow y_t \leftarrow x_t \leftarrow S_t \leftarrow o_t.$$

Unlike continuous dynamical systems with implicit state evolution, ESM advances through explicit commitments at discrete decision boundaries. Time indexes completed reasoning steps, not continuous flow.

Outcomes r_t (including explicit no-op) shape turn $t+1$ via environmental effects or state handoff in layered ESMs. Trajectories $\{x_1, x_2, \dots\}$ emerge from locally governed decisions, not global optimization.

Policy remains the sole mechanism authorizing outcomes, ensuring evolution follows explicit architectural constraints rather than hidden dynamics. Each turn's inspectability enables trajectory analysis without assuming optimal control.

3 The Turn as the Fundamental Unit of Computation

The turn $T_t = (o_t, S_t, x_t, y_t, g_t, r_t)$ is the atomic reasoning object: a complete, inspectable frame committing to state x_t before policy consideration.

State mutation occurs only at turn conclusion via authorized policy outcomes (r_t), including explicit no-op. Each turn constrains subsequent evolution through its committed state and outcome.

- o_t: observations at time t
- S_t: signals computed from o_t
- x_t: committed state vector
- y_t: projection ($P(x_t)$) into policy coordinates (no semantic alteration)
- g_t: gating signal authorizing policy evaluation
- r_t: authorized outcome (no-op, action, escalation, handoff, clarification)

Policy is the sole mechanism producing r_t, activated only when g_t permits. Every turn constitutes a complete, auditable decision record exposing the full causal chain.

Initialization establishes x_0 from structured inputs (intake, prior outputs, baselines), ensuring the first turn begins from explicit context rather than empty state.

Sequences $\{T_1, T_2, \dots\}$ form reconstructible trajectories for analysis. The turn structure is domain-invariant: clinical ESMs use physiological signals/treatment policies; manufacturing uses process signals/control policies. The computational scaffold remains identical.

Generalization arises from reusing this fixed turn structure across domains via redefined, versioned signals/projections/policies, not parameter adaptation within models.

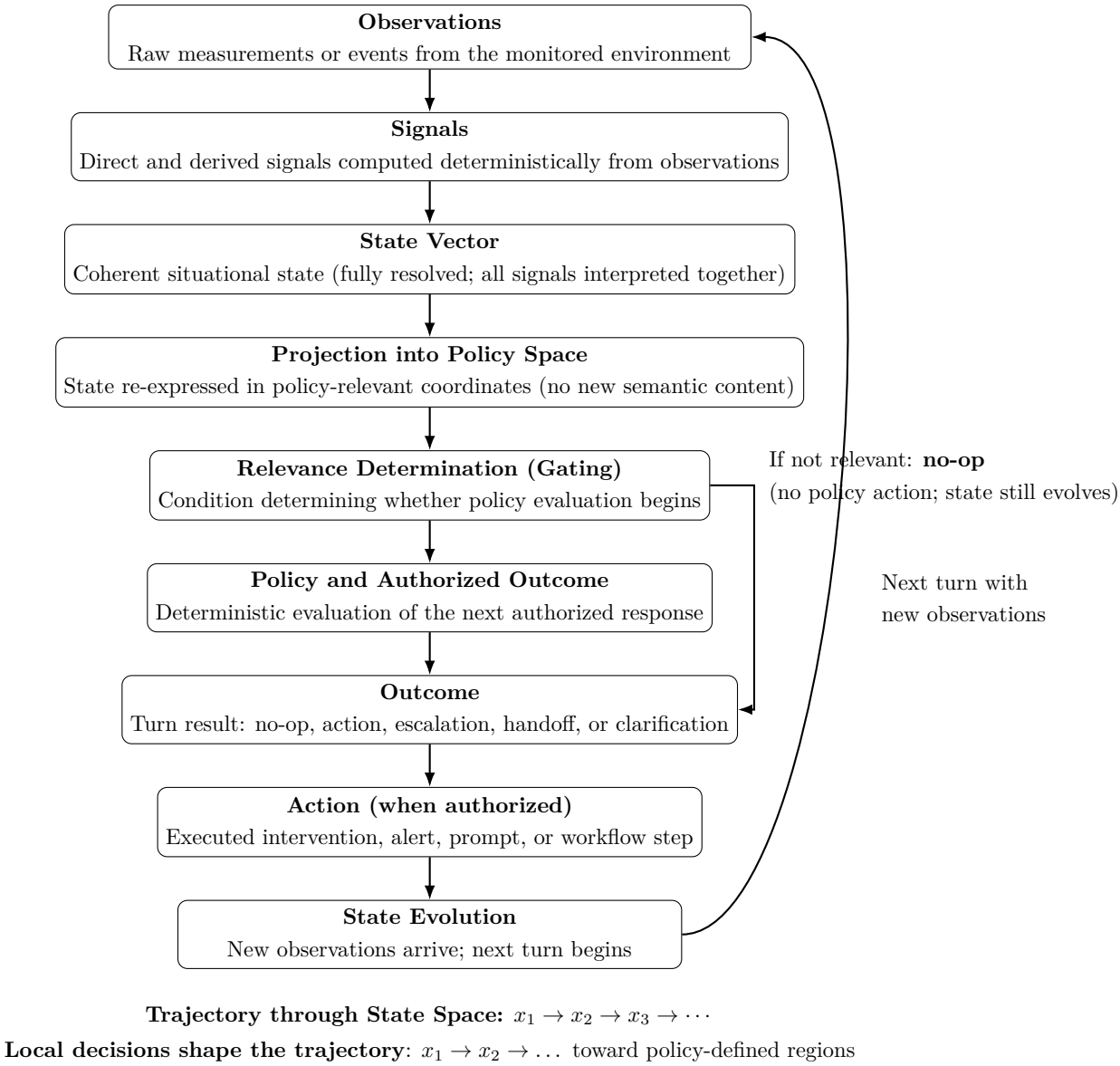

Figure 2: Turn structure: $o_t \to S_t \to x_t \to y_t \to g_t \to r_t$. Policy produces outcomes only when gating authorizes.

4 From Observations to Signals

Signals S_t define the system's representational capacity: explicit, deterministic distinctions over observations o_t that determine what can be known, tracked, and acted upon.

4.1 Direct Signals

Direct signals extract values from o_t:

$$s_i^{\text{direct}} = m_i(o_t) \tag{1}$$

where m_i is a deterministic measurement function (e.g., heart rate, respiratory rate, student response).

4.2 Derived Signals

Derived signals compute relationships over observations or prior signals:

$$s_j^{\text{derived}} = f_j(o_t, o_{t-k:\,t-1}, S_{t-1}) \tag{2}$$

where f_j formalizes domain knowledge (ratios, trends, persistence, shock index $\frac{HR}{SBP}$).

4.3 Signal Set

The complete set is $S_t = S_t^{\text{direct}} \cup S_t^{\text{derived}}$, grounding all subsequent state construction and policy evaluation.

Signals are designed distinctions, not neutral measurements. Their definitions determine representational capacity—what's visible to reasoning is exactly what's captured as signals. Undetected patterns remain invisible by architectural constraint.

Signal design is governance: choices shape detection, relationships, and actionability. Signals are deterministic, versioned, and auditable, enabling controlled evolution via Instrumented Deterministic Evolution (IDE).

S_t forms the explicit interface between raw o_t and committed state x_t, operationalizing domain knowledge as executable structure.

5 Constructing the Committed State

The state x_t is a coherent, committed representation of the current situation, constructed from signals S_t via explicit organizational rules for decision relevance.

Formally,

$$x_t = (s_1, s_2, \ldots, s_n), \quad s_i \in S_t \tag{3}$$

where components are heterogeneous (numeric, categorical, uncertainty indicators) with explicitly known definitions, types, and scales—no vector space operations assumed.

All components describe the same moment, interpreted together as a fully resolved situational construct. Interpretation completes here; no partial estimates or latent inference. Downstream

components operate on this committed representation.

Deterministic construction from versioned signals ensures identical inputs produce identical x_t, enabling audit and replay.

6 Relevance Determination via Gating

Gating $g_t = G(y_t) \in \{0, 1\}$ authorizes policy evaluation over projected state y_t. The system always constructs state, but invokes policy only when $g_t = 1$.

If $g_t = 1$, policy evaluates and produces outcome r_t. If $g_t = 0$, outcome is explicit no-op. Every turn evaluates; not every turn acts.

Gating operates solely on y_t, ensuring consistent decision coordinates. Conditions may include thresholds, persistence, region transitions, volatility, or composite relationships—not limited to single-signal exceedance.

Gating separates continuous state construction from conditional intervention. Explicit G functions are inspectable: any turn's activation can be traced to specific y_t conditions.

Multi-policy configurations use multiple $G_i(y_t)$, preserving deterministic invocation independent of policy logic. Gating establishes invocation determinism: both when decisions occur and their outcomes are governed explicitly.

7 Projection into Policy Space

Projection $y_t = P(x_t)$ re-expresses the committed state x_t in policy-relevant coordinates without introducing new semantic content. Interpretation completes at state construction; projection enables decision evaluation.

P is a deterministic transformation (linear, nonlinear, or model-assisted) whose definition remains explicit and inspectable. Policy operates solely on y_t, never directly on raw signals or unprojected state.

Each y_t component represents a policy-relevant dimension (e.g., respiratory instability, claim strength, confidence, volatility) defined by domain experts. Confidence/uncertainty may be included as explicit coordinates, computed deterministically or via reproducible models.

Projection separates situational description from decision framing: - x_t: what is happening (coherent situational construct) - y_t: how it matters for policy (decision coordinates)

Changes to P revise decision coordinates explicitly via versioned artifacts, preserving inspectability. Gating $G(y_t)$ and policy $\pi(y_t)$ operate in consistent coordinates.

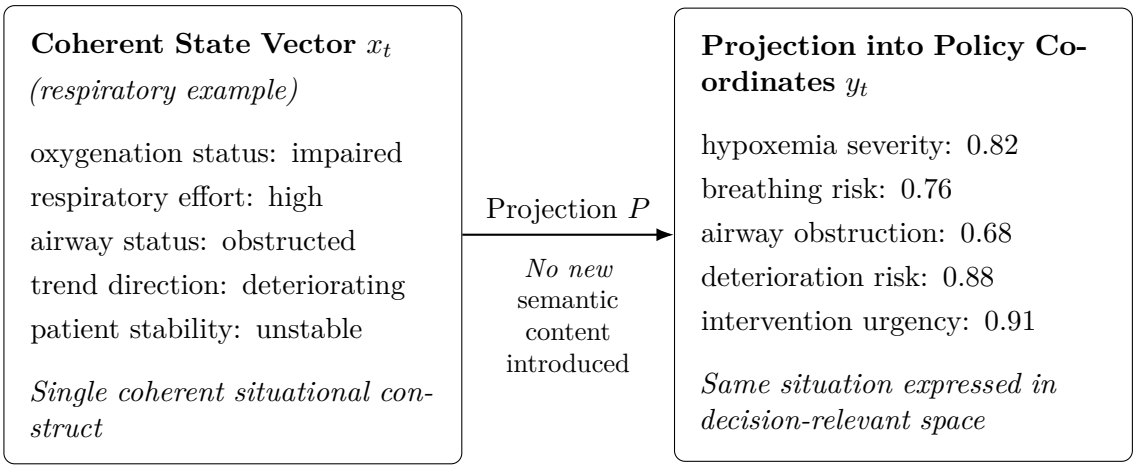

Figure 3: Worked example of state construction and projection in a respiratory ESM. The coherent state vector x_t represents a fully resolved situational construct. Projection maps that same situation into policy-relevant coordinates y_t without introducing new semantic content, enabling downstream relevance evaluation and policy to operate on an explicitly defined decision space.

8 Policy and Authorized Outcomes

Policy $\pi(y_t)$ is the sole mechanism authorizing outcomes r_t, invoked only when gating permits ($g_t = 1$).

$$g_t = 0 \quad \Rightarrow \quad r_t = \text{no-op} \tag{4}$$

$$g_t = 1 \quad \Rightarrow \quad r_t = \pi(y_t) \quad (a_t \in \{\text{action, escalation, handoff, clarification}\}) \tag{5}$$

No-op is explicit: policy evaluated but found no intervention warranted. Policy operates only on projected state y_t, evaluating present conditions via explicit rules—no trajectory optimization or long-horizon planning.

Complete causal chain:

$$r_t \leftarrow g_t \leftarrow y_t \leftarrow x_t \leftarrow S_t \leftarrow o_t \tag{6}$$

Deterministic π ensures identical y_t yields identical r_t. All prior stages establish situation and relevance; only policy authorizes action.

9 State Evolution and Trajectories

State evolution proceeds through successive turns: $x_{t+1} = \Phi(x_t, o_{t+1})$, where Φ deterministically constructs committed state from new observations and explicit context (derived signals,

initialization).

Trajectories form as observable sequences:

$$\{x_1, x_2, \dots\}, \quad \{y_1, y_2, \dots\}, \quad \{T_1, T_2, \dots\} \tag{7}$$

Real-world time between turns varies; temporal structure enters via derived signals (trends, persistence, decay). No continuous dynamics—evolution occurs at discrete decision boundaries.

Trajectories arise from locally governed turns, not global optimization. Policy-defined desirable regions characterize observed patterns (convergence, persistence, divergence) without assuming attractors or optimal control.

Local evaluation constrains evolution to reachable configurations from current committed state, reflecting path dependence without dynamical systems formalism.

10 Layered ESMs and State Handoff

ESMs compose hierarchically: upstream ESMs produce initialized state $x^{(u)}_{\text{handoff}}$ for downstream ESMs.

Downstream initialization:

$$x^{(d)}_1 = H\Big(x^{(u)}_{\text{handoff}},\, o^{(d)}_1\Big) \tag{8}$$

H explicitly transforms upstream state into downstream signals via deterministic rules, preserving auditability. No hidden coupling—state transfers explicitly at turn boundaries.

Key properties:

- **Explicit initialization**: Every ESM begins from defined x_0, internal or upstream
- **Discrete handoff**: Coordination via turn outcomes and state transfer, not continuous sharing
- **Temporal independence**: ESMs operate at different frequencies, synchronized via explicit handoffs
- **Modularity preserved**: Local reasoning + shared context without collapsing authority

Layered ESMs enable specialized reasoning over shared context while maintaining full traceability across abstraction levels and timescales.

11 Policy-Defined Desirable Regions

Domain experts define desirable regions $\mathcal{Y}_D \subset \mathcal{Y}$ within projected state space $\mathcal{Y}$, representing preferred conditions in policy coordinates.

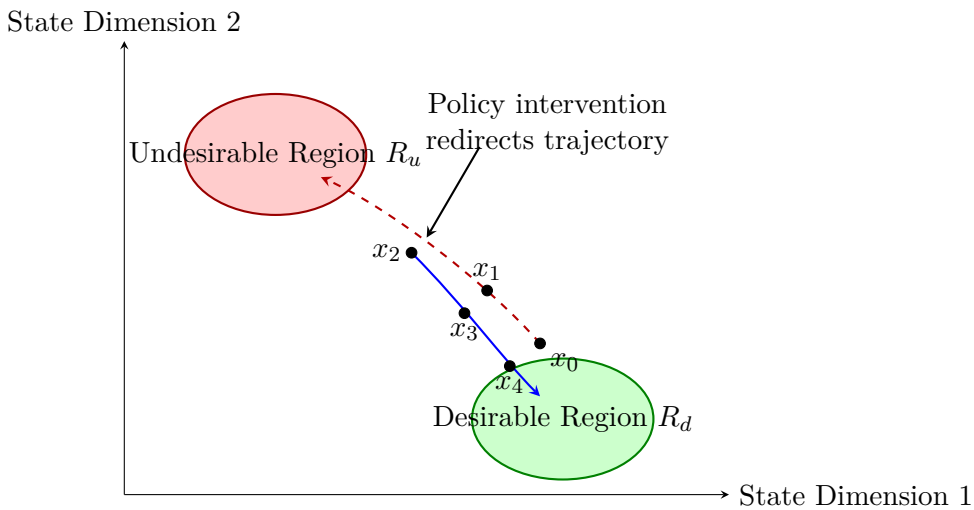

Figure 4: Illustrative trajectory patterns through policy-relevant state space, showing movement relative to desirable and undesirable regions.

$\mathcal{Y}_D$ informs gating/policy design (thresholds, escalation criteria) at design time, not runtime membership checks. Projection P couples to $\mathcal{Y}_D$: changes to P require corresponding region revision.

Trajectories $\{y_1, y_2, \dots\}$ through policy space move toward/away from/persist near $\mathcal{Y}_D$ via world dynamics + policy outcomes. Stability is temporal: sustained residence over turns, not instantaneous points.

Examples:

- Clinical: sustained respiratory stability + low volatility
- Educational: reasoning coherence + evidence alignment over turns

Instrumented Deterministic Evolution (IDE) evaluates signal/projection/gating/policy revisions by trajectory behavior relative to $\mathcal{Y}_D$, enabling systematic refinement without global optimization.

12 Determinism and Auditability

Fixed versioned definitions ensure determinism: identical o_t produces identical r_t via the chain

$$o_t \to S_t \to x_t \to y_t \to g_t \to r_t.$$

12.1 Probabilistic Components

Model-assisted signal extraction/projection outputs become fixed explicit inputs post-capture. Stochasticity is isolated; downstream transformations remain deterministic.

12.2 Replayability

Recorded traces + versioned definitions reproduce complete reasoning trajectories, enabling retrospective analysis of alternative configurations.

12.3 Failure Localization

Undesirable r_t traces to specific stage (signals, state, projection, gating, policy). Layered ESMs localize across handoff boundaries.

12.4 Audit Record

Every turn retains:

- o_t, S_t, x_t, y_t, g_t, r_t
- versioned definitions of all transformations

Audit reconstructs *how* and *under which configuration* decisions occurred, enabling systematic refinement via Instrumented Deterministic Evolution.

13 Governance Boundaries

ESM produces explicit turn records $(o_t, S_t, x_t, y_t, g_t, r_t)$ for direct chain inspection, shifting interpretability from post-hoc explanation to intrinsic property.

13.1 Human Role

Humans define domain semantics:

- signals capturing measurable distinctions
- projections framing decision coordinates
- desirable regions $\mathcal{Y}_D$ (Sec. 11)
- gating thresholds and policy rules π

Teams refine independently:

- clinical: respiratory signals → escalation criteria → intervention policy
- educational: reasoning signals → assessment projections → feedback rules

13.2 AI Role

AI assists interpretation only:

- signal extraction from unstructured data
- derived signal generation
- projection support

AI outputs become fixed explicit inputs. No authority to commit state or authorize action.

13.3 Authority Boundary

Policy $\pi(y_t)$ alone produces r_t when $g_t = 1$. Humans/AI enrich interpretation; policy governs commitment. Failure traces to specific chain stage (Sec. 12).

Versioned artifacts enable targeted revision and human-system co-evolution via Instrumented Deterministic Evolution.

14 Governance through Explicit Artifacts

Governance in ESM targets versioned components: signals, state construction, projection P, gating G, policy π, desirable regions $\mathcal{Y}_D$. These explicit artifacts shape both representation (what's seen) and authority (what's authorized).

Governance spans all layers:

- **Signals**: what distinctions matter (measurement power)
- **Projection**: decision coordinates ($x_t \to y_t$)
- **Gating**: when policy activates ($G(y_t)$)
- **Policy**: what outcomes r_t are authorized
- **Desirable regions**: stability targets $\mathcal{Y}_D$

AI/human contributions (signal extraction, projection support) enter as fixed inputs. Policy $\pi(y_t)$ alone commits state when $g_t = 1$. No implicit updates bypass deterministic boundary.

Stakeholders revise independently:

- refine respiratory signals without changing escalation policy
- adjust $\mathcal{Y}_D$ without modifying gating thresholds
- constrain policy actions without redefining projection

15 Instrumented Deterministic Evolution (IDE)

IDE governs evolution: behavioral change occurs *only* through versioned artifact revision.

15.1 Replay Evaluation

Recorded traces replay under config v:

$$r_t^{(v)} = R^{(v)}(o_t) \quad \text{vs.} \quad r_t^{(v')} = R^{(v')}(o_t)$$

Compare trajectories $\{T_1^{(v)}, \dots\}$ vs $\{T_1^{(v')}, \dots\}$ to assess signal/projection/gating/policy changes.

15.2 Training Loop

1. Record interaction traces
2. Train improved signal extractors/projections offline
3. Replay traces under candidate v'
4. Deploy v' as versioned artifacts if validated

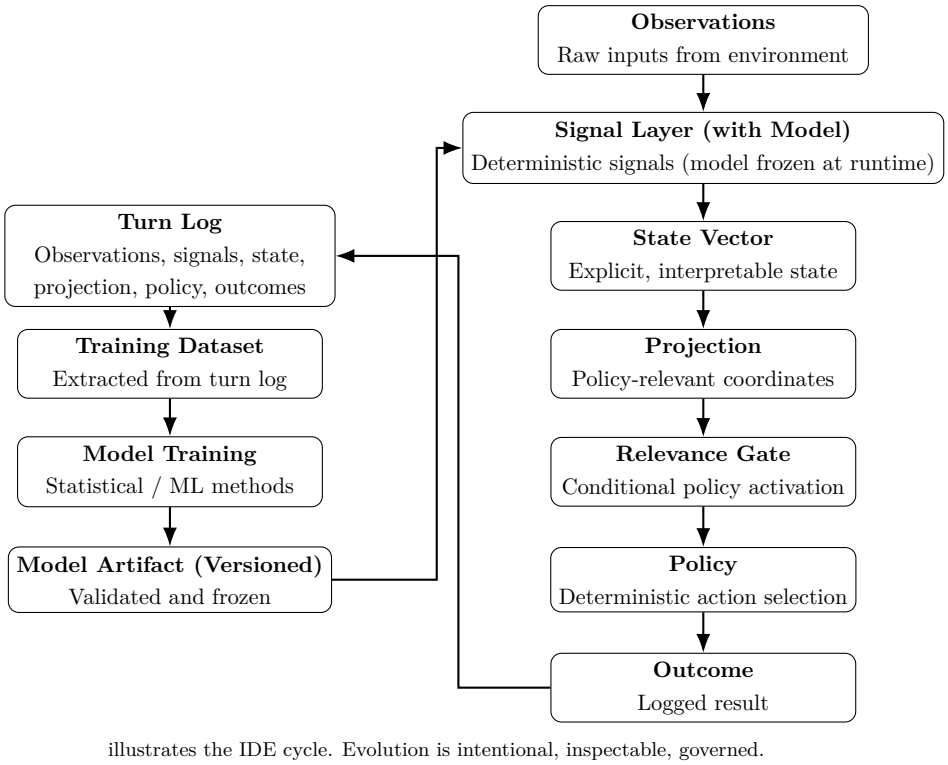

Figure 5: Instrumented Deterministic Evolution: offline refinement of versioned artifacts remains structurally separated from runtime execution.

AI proposes refinements; humans authorize deployment. Policy authority π remains deterministic—learning enriches interpretation, never bypasses decision boundary.

16 Cybernetics Foundations

ESM formalizes cybernetic principles within explicit, governable architecture:

Cybernetics established feedback regulation [10, 1]. ESM extends this by making *signal construction* explicit—cybernetic "distinctions that make a difference" become versioned artifacts.

Second-order cybernetics emphasized observer-defined boundaries [9]. ESM operationalizes this: humans define signals S_t, projections P, desirable regions $\mathcal{Y}_D$, policies π.

Complexity science shows behavior emerges from local interactions [11]. ESM trajectories $\{T_1, T_2, \dots\}$ through committed states realize this via discrete turns, not continuous dynamics.

Key ESM extension: representational structure (signals $\to$ state $\to$ projection) becomes inspectable, versioned, governable. Traditional cybernetics treats perception as given; ESM makes it first-class architecture.

Policy $\pi(y_t)$ retains cybernetic feedback while governance spans full chain $o_t \to S_t \to x_t \to y_t \to g_t \to r_t$. System evolution becomes refinement of both behavior *and* perception.

17 Architecture Boundaries

ESM enforces structural separation but doesn't solve core modeling challenges. It's a decision *architecture*, not a universal reasoning method.

17.1 No Global Optimization

Unlike dynamic programming over trajectories [3], ESM makes *local* decisions: $\pi(y_t)$ evaluates present conditions only. Behavior emerges from turn sequences, not horizon optimization.

17.2 Projection Depends on Signals

Flawed signals $\to$ flawed $x_t \to$ flawed y_t. Deterministic policy faithfully executes poor representations. ESM guarantees *inspectability*, not correctness.

17.3 Policy Design Remains Human

$\pi(y_t)$ encodes normative judgment. Bad thresholds/escalation remain bad, but visible/revisable.

17.4 Partial Observability

No embedded POMDPs/belief states. Ambiguity $\to$ explicit clarification request (policy-triggered) or proceed under defined uncertainty. No probabilistic authority bypass.

17.5 Generative AI Role

LLMs assist signals/projection only $\to$ fixed explicit inputs. Never commit state/authorize r_t. Interpretation $\neq$ authority.

17.6 Deployment Constraints

Safety-critical: restrict A to pre-authorized actions. Humans/supervisors retain final authority. ESM makes *all* outcomes explicit/versioned.

ESM complements learning/optimization as *governance framework* for consequential decisions requiring auditability/accountability.

18 Related Work

ESM synthesizes control theory, cybernetics, and AI governance into explicit turn-based architecture.

Control Theory: Adopts state-space trajectories [2] but replaces latent estimation with explicit signal → state → projection chain. Local policy $\pi(y_t)$ vs global optimization.

System ID/Signal Processing: Explicit signals S_t replace learned features [6]. Representation becomes governed construction, not inference.

Representation Learning: Committed state x_t + projection $y_t = P(x_t)$ vs latent embeddings. Semantic transparency over efficiency.

Decision Theory: Externalizes uncertainty into explicit x_t, y_t coordinates vs POMDP beliefs. Deterministic $\pi(y_t)$ under defined ambiguity.

Naturalistic Decision Making: Turn structure mirrors pattern recognition → policy response [5].

Event-Triggered Control: Gating $g_t = G(y_t)$ generalizes conditional activation [8].

Hierarchical Systems: Layered ESMs via explicit state handoff $x_1^{(d)} = H(x_{\text{handoff}}^{(u)}, o_1^{(d)})$ [7].

Distinguishing ESM Synthesis:

- **Coherent committed state** x_t from explicit signals S_t
- **Projection** $y_t = P(x_t)$: no new semantics
- **Gating** g_t: explicit decision boundary
- **Policy authority**: $\pi(y_t)$ sole r_t producer
- **IDE**: versioned evolution via replay $\{T_i^{(v)} \leftrightarrow T_i^{(v')}\}$

ESM: governance architecture for consequential decisions, complementing (not replacing) learning/optimization.

19 Conclusion

The Emergent State Machine (ESM) defines a deterministic architecture for interpretable situational reasoning through discrete *turns*: $o_t \to S_t \to x_t \to y_t \to g_t \to r_t$.

Key contributions:

- **Coherent committed state** x_t: interpretation completes before decision
- **Projection** $y_t = P(x_t)$: re-expresses without new semantics
- **Gating** $g_t = G(y_t)$: explicit decision boundary

- **Policy authority**: $\pi(y_t)$ sole r_t producer (no-op, action, escalation)
- **Instrumented Deterministic Evolution (IDE)**: versioned artifact revision via replay
- **Layered ESMs**: explicit state handoff preserves modularity/traceability

Trajectories $\{T_1, T_2, \dots\}$ through policy space emerge from local decisions, analyzable via desirable regions $\mathcal{Y}_D$ without global optimization.

ESM doesn't solve modeling challenges but makes them *explicit*: signals, projections, policies remain human-defined, inspectable, revisable. Probabilistic components inform interpretation; policy governs authority.

Reasoning becomes the object of governance. Turns enable analysis, replay, systematic evolution—transparent, accountable systems for consequential domains.

References

[1] W. Ross Ashby. *An Introduction to Cybernetics*. Chapman & Hall, 1956.

[2] Karl Johan Åström and Richard M. Murray. *Feedback Systems: An Introduction for Scientists and Engineers*. Princeton University Press, 2010.

[3] Richard Bellman. *Dynamic Programming*. Princeton University Press, 1957.

[4] Mica R. Endsley. Toward a theory of situation awareness in dynamic systems. *Human Factors*, 37(1):32–64, 1995.

[5] Gary Klein. *Sources of Power: How People Make Decisions*. MIT Press, 1998.

[6] Lennart Ljung. *System Identification: Theory for the User*. Prentice Hall, 1999.

[7] Herbert A. Simon. The architecture of complexity. *Proceedings of the American Philosophical Society*, 106(6):467–482, 1962.

[8] Paulo Tabuada. Event-triggered real-time scheduling of stabilizing control tasks. *IEEE Transactions on Automatic Control*, 52(9):1680–1685, 2007.

[9] Heinz von Foerster. Cybernetics of cybernetics. In *Understanding Understanding*. Springer, 1974.

[10] Norbert Wiener. *Cybernetics*. MIT Press, 1948.

[11] Stephen Wolfram. *A New Kind of Science*. Wolfram Media, 2002.